

Module Interface Specification for Software Engineering

Team #16, The Chill Guys

Hamzah Rawasia

Sarim Zia

Aidan Froggatt

Swesan Pathmanathan

Burhanuddin Kharodawala

April 6, 2026

1 Revision History

Date	Developer	Notes
November 1st	Aidan Froggatt	Completed Section 6.1, 6.2, 6.3, 6.4
November 11th	Swesan Pathmanathan	Completed Sections 1, 2, 4, 5, 6.5, 6.6, 6.7, 6.8
November 12th	Burhan Kharodawala	Completed Sections 3, 6.9, 6.10, 6.11, 6.12
March 29th	Swesan Pathmanathan	Notation (§4): full-width <code>tabularx</code> for primitive types; clarified <code>UpdatedFields</code> without <code>? syntax</code> (peer review #245). Intro (#244): repository URL via <code>\url</code> (visible text, clickable).
April 5th	Sarim Zia	Implemented TA and Peer feedback

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at <https://github.com/hitchly/hitchly/tree/main/docs/SRS>

symbol	description
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DB	Database
DBMS	Database Management System
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Secure Hypertext Transfer Protocol
JWT	JSON Web Token
MFA	Multi-Factor Authentication
MG	Module Guide
MIS	Module Interface Specification
ML	Machine Learning
ORM	Object-Relational Mapping
OTP	One-Time Password
SDK	Software Development Kit
SQL	Structured Query Language
SRS	Software Requirements Specification
SSO	Single Sign-On
UI	User Interface
UX	User Experience
Software Engineering	Hitchly rideshare system name

Contents

1	Revision History	1
2	Symbols, Abbreviations and Acronyms	2
3	Introduction	1
4	Notation	1
5	Module Decomposition	3
6	Authentication & Verification Module	8
6.1	Module	8
6.2	Uses	9
6.3	Syntax	9
6.4	Semantics	9
7	User Profile Module	11
7.1	Module	11
7.2	Uses	12
7.3	Syntax	12
7.4	Semantics	13
8	Route & Trip Module	14
8.1	Module	14
8.2	Uses	15
8.3	Syntax	15
8.4	Semantics	16
9	Matching Module	18
9.1	Module	18
9.2	Formal Specification	19
9.2.1	State Definitions	21
9.2.2	Mathematical Definitions	22
9.3	Uses	23
9.4	Syntax	24
9.5	Semantics	24
10	Scheduling Module	30
10.1	Module	30
10.2	Uses	31
10.3	Syntax	32
10.4	Semantics	32

11 Notification Module	34
11.1 Module	34
11.2 Uses	34
11.3 Syntax	35
11.4 Semantics	35
12 Rating & Feedback Module	36
12.1 Module	36
12.2 Uses	37
12.3 Syntax	37
12.4 Semantics	37
13 Safety & Reporting Module	38
13.1 Module	38
13.2 Uses	38
13.3 Syntax	39
13.4 Semantics	39
14 Payment Module	39
14.1 Module	39
14.2 Uses	40
14.3 Syntax	40
14.4 Semantics	40
15 Database Module	42
15.1 Module	42
15.2 Uses	42
15.3 Syntax	42
15.4 Semantics	43
16 Pricing Module	44
16.1 Module	44
16.2 Uses	45
16.3 Syntax	45
16.4 Semantics	45
17 Admin/Moderation Module	46
17.1 Module	46
17.2 Uses	47
17.3 Syntax	47
17.4 Semantics	47
18 Appendix	48

3 Introduction

The following document details the Module Interface Specifications for Hitchly, a rideshare application designed for the McMaster community. Its main purpose is to provide a reliable and trustworthy platform for McMaster students, staff and faculty members to find and provide rideshare services. It does it through its robust matching algorithm which matches users based on their timetable, location time, and preferences. It aims to make commuting to and from university sustainable and cost-effective.

Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at <https://github.com/hitchly/hitchly>.

4 Notation

The structure of the MIS for modules comes from Hoffman and Strooper (1995), with the addition that template modules have been adapted from Ghezzi et al. (2003). The mathematical notation comes from Chapter 3 of Hoffman and Strooper (1995). For instance, the symbol $:=$ is used for a multiple assignment statement and conditional rules follow the form $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | \dots | c_n \Rightarrow r_n)$.

The following table summarizes the primitive data types used by Software Engineering.

Data Type	Notation	Description
character	char	a single symbol or digit
integer	$\mathbb{Z}$	a number without a fractional component in $(-\infty, \infty)$
natural number	$\mathbb{N}$	a number without a fractional component in $[1, \infty)$
real	$\mathbb{R}$	any number in $(-\infty, \infty)$

The specification of Software Engineering uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Software Engineering uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

The following enumerated and composite types are used throughout the specification:

- **Role** = {rider, driver, both} — User role enumeration
- **Day** = {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday} — Day of week enumeration
- **TripStatus** = {pending, active, completed, canceled} — Trip state enumeration

- **SwipeType** = {like, pass} — Swipe action enumeration
- **Location** = (latitude: $\mathbb{R}$, longitude: $\mathbb{R}$) — Geographic coordinates
- **UserRecord** = (userId: $\mathbb{N}$, name: string, email: string, role: Role, faculty: string, year: $\mathbb{N}$, bio: string, profileImageUrl: string, verified: bool) — Complete user profile
- **UpdatedRecord** = UserRecord — Updated user profile (same structure as UserRecord)
- **PreferenceData** = (userId: $\mathbb{N}$, quietRide: bool, musicPreference: string, smokingAllowed: bool, maxDetour: $\mathbb{R}$) — User ride preferences
- **UpdatedPreference** = PreferenceData — Updated preferences (same structure as PreferenceData)
- **TripRecord** = (tripId: $\mathbb{N}$, driverId: $\mathbb{N}$, origin: Location, destination: Location, departureTime: $\mathbb{R}$, availableSeats: $\mathbb{N}$, status: TripStatus) — Complete trip information
- **TripList** = seq of TripRecord — Sequence of trip records
- **TripData** = (origin: Location, destination: Location, departureTime: $\mathbb{R}$, availableSeats: $\mathbb{N}$) — Trip creation input
- **UpdatedFields** — Partial trip update input: each of origin (Location), destination (Location), departureTime ($\mathbb{R}$), and availableSeats ($\mathbb{N}$) may be present or omitted independently.
- **UpdatedTripRecord** = TripRecord — Updated trip record (same structure as TripRecord)
- **ScheduleRecord** = (scheduleId: $\mathbb{N}$, userId: $\mathbb{N}$, daysOfWeek: seq of Day, time: $\mathbb{R}$, origin: Location, destination: Location, isRecurring: bool) — Recurring schedule definition
- **ScheduleList** = seq of ScheduleRecord — Sequence of schedule records
- **ScheduleData** = (daysOfWeek: seq of Day, time: $\mathbb{R}$, origin: Location, destination: Location, isRecurring: bool) — Schedule creation input
- **RequestData** = (pickup: Location, dropoff: Location, time: $\mathbb{R}$, seatsNeeded: $\mathbb{N}$) — Ride request input
- **MatchRecord** = (matchId: $\mathbb{N}$, riderId: $\mathbb{N}$, driverId: $\mathbb{N}$, tripId: $\mathbb{N}$, score: $\mathbb{R}$, timestamp: $\mathbb{R}$) — Match result with compatibility score
- **RankedMatchList** = seq of MatchRecord — Sequence of match records ordered by score (descending)

- **SwipeRecord** = (swipeId: $\mathbb{N}$, userId: $\mathbb{N}$, targetId: $\mathbb{N}$, swipeType: SwipeType, timestamp: $\mathbb{R}$) — User swipe action record
- **MatchSummary** = (mutualMatches: seq of MatchRecord, pendingSwipes: seq of SwipeRecord) — Complete match status for a user
- **RatingData** = (tripId: $\mathbb{N}$, score: $\mathbb{R}$, comment: string) — Rating submission input
- **RatingRecord** = (ratingId: $\mathbb{N}$, reviewerId: $\mathbb{N}$, revieweeId: $\mathbb{N}$, tripId: $\mathbb{N}$, score: $\mathbb{R}$, comment: string, timestamp: $\mathbb{R}$) — Complete rating information
- **RatingList** = seq of RatingRecord — Sequence of rating records
- **ReportData** = (reportedUserId: $\mathbb{N}$, tripId: $\mathbb{N}$, reason: string, description: string) — Incident report input
- **ReportRecord** = (reportId: $\mathbb{N}$, reporterId: $\mathbb{N}$, reportedUserId: $\mathbb{N}$, tripId: $\mathbb{N}$, reason: string, description: string, status: string, timestamp: $\mathbb{R}$) — Complete incident report
- **ReportList** = seq of ReportRecord — Sequence of report records
- **PaymentRecord** = (paymentId: $\mathbb{N}$, userId: $\mathbb{N}$, tripId: $\mathbb{N}$, amount: $\mathbb{R}$, status: string, timestamp: $\mathbb{R}$) — Transaction record
- **Confirmation** = bool — Operation success confirmation

5 Module Decomposition

Modules are decomposed according to the principle of information hiding ([Parnas et al., 1984](#)). Each module's *Secrets* describes the internal design decision that is intentionally hidden from other modules, while the *Services* field specifies *what* the module provides without revealing *how* the service is implemented. The *Frontend UI*, *API Logic*, and *Database Models* indicate how each module is realized across the system's architecture. *Implemented By* identifies the structural components used, and *Type of Module* classifies each module as a Behaviour-Hiding or Software-Decision module according to Section ???. Only the leaf modules are included.

Behaviour-Hiding Modules

M1: Authentication & Verification Module

- **Secrets:** Token management strategy, identity verification workflow, and credential validation logic.
- **Services:** Allows users to register, log in, and verify institutional identity.

- **Frontend UI:** Authentication interfaces (login, signup, verification).
- **API Logic:** Authentication controller (login, register, verify), session management, and schema validation.
- **Database Models:** User identity entity (credentials, verification status, session tokens).
- **Implemented By:** Client-side authentication flow + server-side identity provider.
- **Type of Module:** Behaviour-Hiding Module.

M2: User Profile Module

- **Secrets:** Role assignment logic and profile data storage structure.
- **Services:** Provides operations for profile editing, preference management, and vehicle registration.
- **Frontend UI:** Profile management view, edit forms, vehicle input screens.
- **API Logic:** User profile controller for CRUD operations.
- **Database Models:** User profile entity, vehicle entity, preference entity.
- **Implemented By:** Client-side forms + server-side profile manager.
- **Type of Module:** Behaviour-Hiding Module.

M3: Route & Trip Module

- **Secrets:** Trip data schema, route normalization logic, and timestamp formatting.
- **Services:** Allows users to create, query, and cancel trip records.
- **Frontend UI:** Trip creation interface, trip listing view, route selection component.
- **API Logic:** Trip controller for lifecycle management (create, read, delete).
- **Database Models:** Trip entity (origin, destination, timestamp, capacity).
- **Implemented By:** Client-side trip manager + server-side route handler.
- **Type of Module:** Behaviour-Hiding Module.

M6: Notification Module

- **Secrets:** Delivery mechanism details and asynchronous event queue handling.
- **Services:** Dispatches alerts for system events (matches, cancellations, reminders).
- **Frontend UI:** Notification center view, device-level push integration.
- **API Logic:** Notification dispatcher and event listener service.
- **Database Models:** Notification log entity.
- **Implemented By:** External push service interface + server-side dispatcher.
- **Type of Module:** Behaviour-Hiding Module.

M7: Rating & Feedback Module

- **Secrets:** Reputation scoring formula and post-ride feedback aggregation logic.
- **Services:** Collects and processes user-to-user star ratings and textual reviews strictly for calculating rider/driver reliability scores.
- **Frontend UI:** Feedback submission screen.
- **API Logic:** Rating controller.
- **Database Models:** Rating entity (linked to match context).
- **Implemented By:** Client-side feedback form + server-side reputation manager.
- **Type of Module:** Behaviour-Hiding Module.

M8: Safety & Reporting Module

- **Secrets:** Incident data collection formatting and automated severity-flagging thresholds.
- **Services:** Captures user-submitted safety reports and severe incident tickets, securely storing them for future administrative review. (This module does not handle the review or banning process).
- **Frontend UI:** Report submission interface, safety resource view.
- **API Logic:** Report handling service.
- **Database Models:** Incident report entity.
- **Implemented By:** Client-side reporting tool + server-side safety logic.
- **Type of Module:** Behaviour-Hiding Module.

M9: Payment & Cost Estimation Module

- **Secrets:** Fare calculation rules and transaction validation logic.
- **Services:** Estimates service costs and records transaction states.
- **Frontend UI:** Cost display, confirmation interface.
- **API Logic:** Payment controller.
- **Database Models:** Transaction entity, payment record.
- **Implemented By:** Server-side logic + transaction ledger.
- **Type of Module:** Behaviour-Hiding Module.

M10: Admin & Moderation Module

- **Secrets:** Moderation enforcement policies, user ban execution logic, and platform-wide analytics aggregation.
- **Services:** Provides administrative oversight by retrieving data collected by the Safety module, executing user bans, managing role permissions, and generating system-wide analytics.
- **Frontend UI:** Administrative dashboard (optional).
- **API Logic:** Admin controller.
- **Database Models:** Audit logs, restriction lists.
- **Implemented By:** Server-side admin tools.
- **Type of Module:** Behaviour-Hiding Module.

Software-Decision Modules

M4: Matching Module

- **Secrets:** Compatibility scoring algorithm, parameter weighting, and ranking strategy.
- **Services:** Computes compatibility between entities and returns ordered results.
- **Frontend UI:** Match presentation interface (card stack or list view).
- **API Logic:** Matchmaking service and algorithm implementation.
- **Database Models:** Match result entity, interaction entity (swipes/decisions).
- **Implemented By:** Server-side algorithm engine + heuristics library.
- **Type of Module:** Software-Decision Module.

M5: Scheduling Module

- **Secrets:** Recurring event generation logic and temporal parsing rules.
- **Services:** Generates and manages recurring or single-instance schedule entries.
- **Frontend UI:** Scheduling interface (time picker, recurrence settings).
- **API Logic:** Schedule parser and generation service.
- **Database Models:** Schedule entity.
- **Implemented By:** Server-side scheduling engine.
- **Type of Module:** Software-Decision Module.

M11: Pricing Module

- **Secrets:** Cost constants (rates, multipliers) and pricing model structure.
- **Services:** Computes financial estimates based on distance and service parameters.
- **Frontend UI:** Cost preview component.
- **API Logic:** Pricing calculation service (shared utility).
- **Database Models:** Pricing configuration parameters.
- **Implemented By:** Shared utility library.
- **Type of Module:** Software-Decision Module.

M12: Database Module

- **Secrets:** Data persistence implementation, schema definitions, and query optimization.
- **Services:** Provides a unified interface for data storage and retrieval.
- **Frontend UI:** None.
- **API Logic:** Database client configuration and entity exports.
- **Database Models:** All system entities.
- **Implemented By:** Object-Relational Mapper (ORM) + Relational Database System.
- **Type of Module:** Software-Decision Module.

6 Authentication & Verification Module

6.1 Module

The Authentication & Verification Module manages user identity creation, secure login, and institutional email verification for all system users. This module is implemented using a secure **Authentication Framework**, which provides end-to-end session management, credential validation, and token-based verification. It ensures that only users with verified institutional emails (e.g., @mcmaster.ca) can access the system's features.

Frontend The frontend component is integrated into the client application. It provides the user interface flow for login, signup, and verification. User input is collected through secure forms and transmitted to the backend via remote procedure calls. The frontend handles the following:

- Displays login and signup screens.
- Manages form validation and field state.
- Invokes authentication client hooks for session management.
- Handles redirection and session persistence after successful login.
- Presents error messages for invalid credentials or unverified accounts.

Backend The backend is implemented in the application server using the framework's server-side SDK. It manages credential validation, session tokens, domain enforcement, and email verification logic. Key backend operations include:

- Configuring the identity provider for the API.
- Enforcing allowed email domains during registration.
- Generating and validating secure verification tokens.
- Managing persistent user sessions through secure tokens.
- Integrating with the email service for verification links.

Data The data layer persists authentication and verification information within the relational database managed by the persistence layer. The module automatically provisions and maintains required entities:

- **UserIdentity** — stores credentials, roles, and verification state.
- **SessionStore** — stores active user sessions with expiry timestamps.

- **VerificationTokens** — tracks issued and redeemed email verification tokens.

Data integrity is enforced by unique constraints on email addresses and transactional updates during verification.

6.2 Uses

This module ensures secure access to the system by verifying each user's institutional affiliation. It interacts directly with:

- **Frontend UI:** collects credentials and displays verification flows.
- **API Layer:** provides endpoints for signup, login, logout, and verification.
- **Database:** persists user and session data.

All other modules depend on this component to validate user identity before allowing access to protected features.

6.3 Syntax

Exported Constants

- **ALLOWED_DOMAIN** = "@mcmaster.ca" Restricts signup to allowed institutional domains.
- **SESSION_EXPIRY** = 86400 Sets the maximum lifetime (in seconds) for user sessions.

Exported Access Programs

Name	In	Out	Exceptions
<code>registerUser</code>	name: string, email: string, password: string	token: string	DuplicateEmail, InvalidDomain
<code>loginUser</code>	email: string, password: string	session: string	InvalidCredentials
<code>verifyEmail</code>	token: string	success: bool	ExpiredToken
<code>logoutUser</code>	session: string	success: bool	InvalidSession

6.4 Semantics

State Variables

- **Users:** Set of `UserRecord` — The collection of all registered users and their states.
- **Sessions:** Set of `SessionRecord` — The collection of currently active sessions.
- **PendingVerifications:** Set of (token: string, email: string) — Map of active verification tokens.

Environment Variables

- User input device (keyboard, network interface).
- Secure transport channel (HTTPS).
- External Email Service (for delivery of verification codes).

Assumptions

- Users possess valid institutional email accounts.
- The persistence layer is initialized and reachable.
- Network connectivity is stable during the authentication handshake.

Access Routine Semantics `registerUser(name, email, password):`

- **transition:** Adds a new entry to `Users` with `verified = false`. Generates a unique token t and adds $(t, email)$ to `PendingVerifications`.
- **output:** Returns t (to be sent via email).
- **exception:** `DuplicateEmail` if $email \in Users$. `InvalidDomain` if email does not end with `ALLOWED_DOMAIN`.

`loginUser(email, password):`

- **transition:** Verifies credentials against `Users`. If valid, creates a new session s and adds to `Sessions`.
- **output:** Returns session token s .
- **exception:** `InvalidCredentials` if email not found or password hash mismatch.

`verifyEmail(token):`

- **transition:** Checks if $token \in PendingVerifications$. If found, sets corresponding user in `Users` to `verified = true` and removes token from `PendingVerifications`.
- **output:** `true` on success.
- **exception:** `ExpiredToken` if token not found or time-to-live exceeded.

`logoutUser(session):`

- **transition:** Removes $session$ from `Sessions`.
- **output:** `true`.
- **exception:** `InvalidSession` if session is not active.

Local Functions

- `generateVerificationToken(): string` — Generates a cryptographically secure random string.
- `validateEmailDomain(email: string): boolean` — Returns (*email* endsWith `ALLOWED_DOMAIN`).
- `hashPassword(password: string): string` — Applies a one-way cryptographic hash function to the input.

7 User Profile Module

7.1 Module

The User Profile Module manages all personal and contextual information about system users. It allows students, alumni, and faculty to view and edit their personal data, preferences, and role assignment (rider, driver, or both). This module integrates directly with the Authentication & Verification Module for identity linkage and with the Matching Module to supply accurate attribute data for compatibility scoring.

Frontend The frontend portion resides in the client application. It provides interfaces for profile creation, editing, and data visualization, implemented using modular UI components. Key features include:

- Editable profile screen showing personal attributes, faculty affiliation, and role.
- Driver-specific section for vehicle information (capacity, specifications).
- Preference configuration for ride comfort options (e.g., conversation, music).
- Integration with device media services for optional profile photo management.
- Input validation and error messaging for data integrity.
- Communication with the backend via secure remote procedure calls.

Backend The backend implements the business logic and data orchestration layers. It exposes endpoints for retrieving, updating, and deleting user data entities. Main responsibilities include:

- Providing interface methods for profile retrieval and modification.
- Validating incoming data payloads against strict schema definitions.
- Enforcing access control through session validation.
- Maintaining referential integrity with linked entities (e.g., trips, ratings).
- Broadcasting profile state changes to dependent modules.

Data The data layer defines persistent entities stored in the relational database management system. Key entities include:

- **UserEntity** — stores metadata such as name, role, faculty, and resource locators.
- **VehicleEntity** — stores driver vehicle details and seating capacity.
- **PreferenceEntity** — stores personal ride preferences and configuration settings.

All entities utilize foreign-key constraints to maintain cardinality relationships with user records. Schema migrations ensure consistent structure across deployment environments.

7.2 Uses

This module is used to manage user data required across the entire system. It interacts with:

- **Authentication & Verification Module** - links verified user identity to profile records.
- **Matching Module** - provides profile and preference data for scoring algorithms.
- **Route & Trip Module** - associates user profiles with created or joined trips.
- **Rating & Feedback Module** - aggregates ride feedback to display reliability metrics.

Through these integrations, the module forms the foundation of personalization and trust within the system.

7.3 Syntax

Exported Constants

- **MAX_BIO_LENGTH** = 250 - Character limit for user biography text.
- **DEFAULT_ROLE** = **Role.Rider** - Default role assigned upon registration.

Exported Access Programs

Name	In	Out	Exceptions
<code>getUserProfile</code>	<code>userId: N</code>	<code>UserRecord</code>	<code>NotFound</code>
<code>updateUserProfile</code>	<code>userId: N, data: UpdatedRecord</code>	<code>UserRecord</code>	<code>ValidationFailed</code>
<code>deleteUser</code>	<code>userId: N</code>	<code>Confirmation</code>	<code>Unauthorized</code>
<code>getUserPreferences</code>	<code>userId: N</code>	<code>PreferenceData</code>	<code>NotFound</code>
<code>updatePreferences</code>	<code>userId: N, data: UpdatedPreference</code>	<code>PreferenceData</code>	<code>ValidationFailed</code>

7.4 Semantics

State Variables

- **Profiles:** Map of $(\mathbb{N} \rightarrow \text{UserRecord})$ — The collection of all user profiles indexed by `userId`.
- **Preferences:** Map of $(\mathbb{N} \rightarrow \text{PreferenceData})$ — The collection of user preferences indexed by `userId`.
- **Vehicles:** Map of $(\mathbb{N} \rightarrow \text{VehicleData})$ — The collection of vehicle details indexed by driver `userId`.

Environment Variables

- Client input interface (form data).
- Secure transport channel (HTTPS).
- Persistence layer connection.

Assumptions

- The user is authenticated and identity is verified before modifying profile data.
- The persistence layer schema matches the entity definitions.
- Basic input formatting (sanitization) is performed by the interface layer.

Access Routine Semantics `getUserProfile(userId):`

- **transition:** None.
- **output:** Returns `Profiles[userId]`.
- **exception:** `NotFoundError` if `userId  $\notin$  Profiles`.

`updateUserProfile(userId, data):`

- **transition:** `Profiles[userId] := Profiles[userId]  $\cup$  data` (fields in `data` overwrite existing).
- **output:** Returns updated `Profiles[userId]`.
- **exception:** `ValidationFailed` if `validateProfileInput(data) = false`.

`updatePreferences(userId, data):`

- **transition:** `Preferences[userId] := data`.

- **output:** Returns `Preferences[userId]`.
- **exception:** `ValidationFailed` if `validatePreferenceInput(data) = false`.

`deleteUser(userId)`:

- **transition:** Removes `userId` from `Profiles`, `Preferences`, and `Vehicles`.
- **output:** `true`.
- **exception:** `UnauthorizedError` if `requestorId  $\neq$  userId  $\wedge$  requestorRole  $\neq$  admin`.

Local Functions

- `sanitizeProfileInput(data: UserRecord): UserRecord` — Strips disallowed characters and formatting.
- `mergePreferenceDefaults(prefs: PreferenceData): PreferenceData` — Fills missing fields with system defaults.
- `calculateReliabilityScore(userId: N):  $\mathbb{R}$` — Computes trust metric based on historical interaction data.

8 Route & Trip Module

8.1 Module

The Route & Trip Module manages trip lifecycle events, storage, and retrieval for the system. It enables users to publish, query, and modify trip listings that include origin, destination, departure time, and capacity. This module provides the authoritative source of spatial and temporal data used by the Matching Module for compatibility analysis.

Frontend Implemented within the client application, the frontend presents interactive interfaces for trip management. Key features include:

- Input forms for trip definition (origin, destination, date, time).
- Visual representation of routes using an external mapping service.
- List and detail views for scheduled, active, and completed trips.
- Interface elements for seat management and cancellation.
- Input validation for mandatory fields and logical time constraints.
- Communication with the backend via secure data bindings.

Backend The backend implements business logic and trip operations through defined service endpoints. It is responsible for managing trip state transitions and enforcing data integrity. Primary responsibilities include:

- Exposing interface methods for trip creation, retrieval, and deletion.
- Validating parameters (geographic bounds, capacity limits, temporal logic).
- Associating trips with authenticated user identities.
- Enforcing role-based access control (e.g., driver privileges).
- Broadcasting state changes to dependent modules (Matching, Scheduling).

Data Trip data is persisted in the relational database management system. The schema supports referential integrity between users, trips, and derived entities. Primary entities include:

- **TripEntity** — stores metadata: origin, destination, timestamps, capacity, and driver reference.
- **RequestEntity** — records ride intent from passengers.
- **RouteEntity** — caches geometry and distance metrics for optimization.

Each record maintains audit timestamps, and all modifications are validated against foreign key constraints referencing the user entity.

8.2 Uses

This module is used by the system to manage the core domain object: the trip. It interacts directly with:

- **User Profile Module** — links trips to driver and rider profiles.
- **Matching Module** — provides trip data for compatibility algorithms.
- **Scheduling Module** — integrates recurring patterns into discrete trip entries.
- **Notification Module** — triggers alerts for status changes.

It serves as the foundation for dynamic route pairing and coordination within the system.

8.3 Syntax

Exported Constants

- **MAX_SEATS** = 5 — Maximum allowable passenger capacity per vehicle.
- **TIME_WINDOW_MIN** = 15 — Minimum lead time (in minutes) for trip creation relative to current system time.

Exported Access Programs

Name	In	Out	Exceptions
<code>createTrip</code>	<code>userId: ℕ, data: TripData</code>	<code>TripRecord</code>	<code>ValidationFailed</code>
<code>getTrips</code>	<code>userId: ℕ</code>	<code>TripList</code>	<code>NotFoundError</code>
<code>cancelTrip</code>	<code>userId: ℕ, tripId: ℕ</code>	<code>Confirmation</code>	<code>NotFoundError, UnauthorizedError</code>
<code>updateTrip</code>	<code>userId: ℕ, tripId: ℕ, data: UpdatedFields</code>	<code>UpdatedTripRecord</code>	<code>NotFoundError, UnauthorizedError, ValidationFailed</code>
<code>getTripById</code>	<code>tripId: ℕ</code>	<code>TripRecord</code>	<code>NotFoundError</code>

8.4 Semantics

State Variables

- **Trips:** Map of $(\mathbb{N} \rightarrow \text{TripRecord})$ — The system-wide collection of all trip records indexed by `tripId`.
- **Requests:** Map of $(\mathbb{N} \rightarrow \text{RequestRecord})$ — The collection of pending ride requests.

Environment Variables

- Geolocation hardware (GPS) and external mapping API for spatial route validation.
- System clock for temporal validation against `TIME_WINDOW_MIN`.
- Persistence layer (Database) for reliable storage.

Assumptions

- The user invoking trip creation or modification is authenticated by the Authentication Module.
- The caller requesting trip creation possesses the necessary role privileges (e.g., `Driver`).
- The external mapping service is highly available for real-time route geometry validation.

Access Routine Semantics `createTrip(userId, data):`

- **transition:** Generates a unique `id`. Formats a new `TripRecord` using the provided `userId` as `driverId`, sets `status := active`, and assigns properties from `data`. `Trips[id] := new record`.
- **output:** Returns `Trips[id]`.
- **exception:** `ValidationFailed` if `data.availableSeats > MAX_SEATS`, if coordinates are structurally invalid, or if `data.departureTime < (current time + TIME_WINDOW_MIN)`.

`getTrips(userId):`

- **transition:** None.
- **output:** Returns the sequence $\{t \in \text{Trips} \mid t.\text{driverId} = \text{userId}\}$.
- **exception:** `NotFoundError` if the resulting sequence is empty.

`updateTrip(userId, tripId, data):`

- **transition:** `Trips[tripId] := Trips[tripId]  $\cup$  data` (provided fields in `data` overwrite existing fields).
- **output:** Returns updated `Trips[tripId]`.
- **exception:** `NotFoundError` if `tripId  $\notin$  Trips`. `UnauthorizedError` if `Trips[tripId].driverId  $\neq$  userId`. `ValidationFailed` if `data` introduces invalid coordinates, negative seats, or invalid timestamps.

`cancelTrip(userId, tripId):`

- **transition:** `Trips[tripId].status := canceled`. Dispatches an event to the Notification Module to alert matched riders.
- **output:** Returns `true`.
- **exception:** `NotFoundError` if `tripId  $\notin$  Trips`. `UnauthorizedError` if `Trips[tripId].driverId  $\neq$  userId`.

`getTripById(tripId):`

- **transition:** None.
- **output:** Returns `Trips[tripId]`.
- **exception:** `NotFoundError` if `tripId  $\notin$  Trips`.

Local Functions

- `calculateRouteDistance(origin: Location, destination: Location):  $\mathbb{R}$` — Computes geodesic or routed distance between two coordinate pairs using external mapping caches.
- `validateTripInput(data: TripData): boolean` — Evaluates the logical consistency of input fields, ensuring they fall within permitted thresholds.
- `filterTripsByTime(trips: TripList, startTime:  $\mathbb{R}$ , endTime:  $\mathbb{R}$ ): TripList` — Filters a sequence of trips, returning only those where `departureTime` falls within the temporal bounds $[\text{startTime}, \text{endTime}]$.

9 Matching Module

9.1 Module

The Matching Module is the core algorithmic engine of the system. It is responsible for generating ranked matches between riders and drivers based on route proximity, time compatibility, and ride preferences. This module applies the **Strategy Pattern** to maintain flexibility, allowing future replacement or enhancement of the scoring algorithm (e.g., machine learning or context-aware scoring). Its outputs drive the user interface, where users browse and select potential ride matches.

Frontend Implemented in the client application, the frontend presents matches as interactive cards in a sequential interface. Key features include:

- Input gestures for accepting or passing on potential matches.
- Real-time display of top recommended drivers or riders with route, time, and compatibility score.
- API integration through remote procedure calls.
- Visual match indicators such as qualitative labels or percentage score badges.
- Immediate visual feedback for mutual matches.

Backend The backend implements the matchmaking logic within the application server, encapsulated in a dedicated service component. The design follows the Strategy Pattern to keep scoring algorithms modular. Primary responsibilities include:

- Retrieving candidate users and associated trips from the persistence layer.
- Filtering incompatible pairs by hard constraints: verified institutional email, opposite roles, overlapping time windows, and nearby routes.
- Applying a scoring strategy (default: `WeightedMatchStrategy`) that computes a match score (0–100) using normalized factors.
- Sorting and returning the highest-scoring results to the client.
- Storing selection actions and updating mutual matches.

The backend exposes endpoints for finding matches, submitting selections, and retrieving confirmed results.

Data The data layer maintains persistent records of matches, interactions, and scoring metrics within the relational database. Key entities include:

- **MatchEntity** — stores confirmed mutual matches (rider i - j , driver) with timestamp and score.
- **InteractionEntity** — records all user decisions (accept/pass) for analytics and recommender tuning.
- **MatchHistoryEntity** — archives past matches for feedback and trust scoring.

Foreign keys link match records to both users and trips, ensuring referential integrity.

9.2 Formal Specification

This subsection provides the comprehensive formal specification for the Matching Module using finite state machines and discrete mathematics. The Matching Module is specified as three focused state machines that model distinct aspects of the matching algorithm: match discovery, ride request management, and match viewing.

The state machines are designed to be:

- **Conceptually clear:** Each FSM models a single, well-defined workflow
- **Implementation-ready:** The runtime FSM enforces all edge cases and error handling
- **Documentation-friendly:** Simplified models for specification clarity

Note on Implementation vs. Specification The implementation-level FSM contains additional transitions for error handling, retries, and recovery paths that ensure runtime correctness. For clarity, the documented state machines present simplified conceptual models that focus on the primary workflow, while the runtime FSM enforces all edge cases. This separation follows best practices in formal specification, where abstraction aids understanding while implementation ensures robustness.

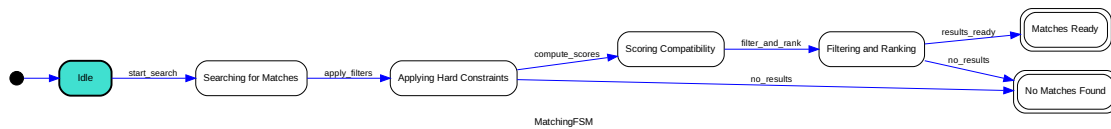


Figure 1: Matching FSM - Handles match discovery, filtering, scoring, and ranking

The Matching FSM (Figure 1) models the core match discovery workflow:

- Searching for potential candidates

- Applying hard constraints (verified email, role compatibility, time windows, routes, seat availability)
- Scoring compatibility using weighted factors
- Filtering and ranking results
- Presenting matches or indicating no matches found

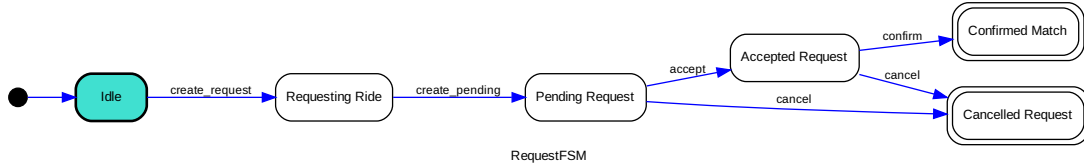


Figure 2: Request FSM - Handles ride request lifecycle

The Request FSM (Figure 2) models the ride request lifecycle:

- Creating a ride request
- Managing pending requests awaiting driver action
- Processing accepted requests
- Confirming matches or handling cancellations

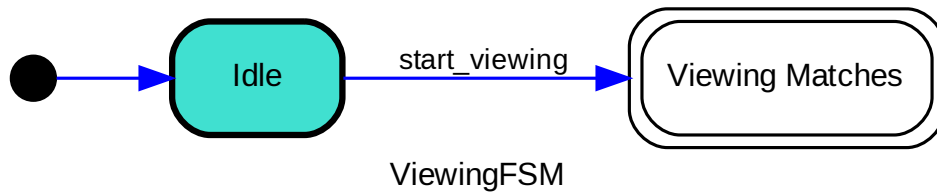


Figure 3: Viewing FSM - Handles viewing confirmed matches

The Viewing FSM (Figure 3) models the simple workflow for viewing confirmed matches:

- Querying accepted ride requests
- Displaying associated driver/rider details

9.2.1 State Definitions

The matching algorithm is modeled as three focused state machines, each handling a distinct aspect of the workflow. This separation improves clarity and maintainability while ensuring each FSM remains conceptually simple.

Matching FSM States The Matching FSM handles the core match discovery and ranking process:

- **Idle:** Initial state, waiting for match discovery request.
- **Searching for Matches:** Retrieving potential candidates from database.
- **Applying Hard Constraints:** Filtering candidates using non-negotiable rules:
 - Verified institutional email
 - Opposite role (rider $\leftrightarrow$ driver)
 - Overlapping time windows
 - Nearby / overlapping routes
 - Sufficient seat availability
- **Scoring Compatibility:** Computing match scores using weighted factors (Schedule, Location, Cost, Comfort, Preferences) and normalizing to 0–100.
- **Filtering and Ranking:** Removing matches below 30% threshold, sorting by score, selecting top 20 results.
- **Matches Ready:** Final state indicating ranked matches are available for presentation.
- **No Matches Found:** Final state indicating no valid matches exist after filtering.

Request FSM States The Request FSM handles the ride request lifecycle:

- **Idle:** Initial state, no active request.
- **Requesting Ride:** Validating ride existence and seat availability, creating request with status `pending`.
- **Pending Request:** Request awaiting driver action (accept/reject).
- **Accepted Request:** Request approved by driver, verifying authorization and updating capacity.
- **Confirmed Match:** Final state indicating mutual match confirmed.
- **Cancelled Request:** Final state indicating request was cancelled (by rider or driver).

Viewing FSM States The Viewing FSM handles viewing confirmed matches:

- **Idle:** Initial state, not viewing matches.
- **Viewing Matches:** Final state where system queries and displays accepted ride requests with associated driver/rider details.

State Machine Interactions While the three FSMs are conceptually separate, they interact in the following ways:

- The Matching FSM produces results that can trigger the Request FSM (when user selects a match).
- The Request FSM produces confirmed matches that can be viewed via the Viewing FSM.
- All FSMs can return to their respective Idle states, allowing the system to handle multiple operations sequentially.
- Error handling and recovery are managed at the implementation level, ensuring robustness while keeping the specification models clean.

9.2.2 Mathematical Definitions

The candidate filtering is formally specified using set theory. Let R be the set of all rides, and let r_r be the rider request. The set of candidate rides is defined as:

$$\begin{aligned} \text{Candidates} = \{r \in R \mid & \text{status}(r) = \text{"scheduled"} \wedge \\ & \text{maxSeats}(r) \geq r_r.\text{maxOccupancy} \wedge \\ & \text{availableSeats}(r) \geq r_r.\text{maxOccupancy}\} \end{aligned} \quad (1)$$

The filtered candidates set excludes rides with compatibility dealbreakers:

$$\text{Filtered} = \{c \in \text{Candidates} \mid \text{compatibilityScore}(c, r_r) > 0\} \quad (2)$$

The scoring function **computeScore** is formally defined as a weighted sum of normalized factors. For the default preference preset, the weights are:

$$\begin{aligned} \text{rawScore}(r, d) = & 2.0 \times \text{scheduleScore}(r.\text{desiredArrivalTime}, d.\text{startTime}) + \\ & 2.0 \times \text{locationScore}(r.\text{origin}, r.\text{destination}, d.\text{origin}, d.\text{destination}) + \\ & 1.5 \times \text{costScore}(r.\text{estimatedCost}, \text{minCost}) + \\ & 0.5 \times \text{comfortScore}(d.\text{currentPassengers}, d.\text{maxSeats}, r.\text{maxOccupancy}) + \\ & 1.0 \times \text{compatibilityScore}(r.\text{prefs}, d.\text{prefs}) \end{aligned} \quad (3)$$

where each factor function maps to the interval $[0, 1]$:

- $\text{scheduleScore} : \text{string} \times \text{string} \rightarrow [0, 1]$ — Computes time compatibility. Returns 1.0 if driver departs 0-20 minutes after rider's desired time, decreasing linearly for larger differences.
- $\text{locationScore} : \text{Location}^4 \rightarrow [0, 1]$ — Computes route compatibility based on detour time. Returns 1.0 if detour is within tolerance, decreasing exponentially for excess detour time.
- $\text{costScore} : \mathbb{R} \times \mathbb{R} \rightarrow [0, 1]$ — Computes cost attractiveness relative to minimum cost among all candidates. Lower cost relative to minimum yields higher score.
- $\text{comfortScore} : \mathbb{N}^3 \rightarrow [0, 1]$ — Computes seat availability comfort. Returns 0.0 if insufficient seats, otherwise $1.0 - \text{currentPassengers}/\text{maxSeats}$.
- $\text{compatibilityScore} : \text{PreferenceData}^2 \rightarrow [0, 1]$ — Computes preference compatibility. Returns 0.0 for dealbreakers (smoking, pets mismatches), otherwise computes soft matching score based on music and chatty preferences.

The raw score is normalized to a percentage (0-100) using the maximum theoretical score:

$$\text{normalizeScore}(\text{rawScore}) = \min \left(100, \left\lfloor \frac{\text{rawScore}}{7.0} \times 100 + 0.5 \right\rfloor \right) \quad (4)$$

where the maximum theoretical score is 7.0 (sum of all weights: $2.0 + 2.0 + 1.5 + 0.5 + 1.0 = 7.0$). The addition of 0.5 before flooring implements rounding to the nearest integer, equivalent to $\text{round}(\text{rawScore}/7.0 \times 100)$.

The final ranked match list is constructed by filtering matches above the threshold, sorting by normalized score, and taking the top candidates:

$$\begin{aligned} \text{validMatches} &= \{m \in \text{scoredMatches} \mid \\ &\quad \text{normalizeScore}(m.\text{rawScore}) \geq \text{MATCH_THRESHOLD} \times 100\} \\ \text{matchResults} &= \text{take}(\text{sort}(\text{validMatches}, \text{descending}), \text{MAX_CANDIDATES}) \end{aligned} \quad (5)$$

where $\text{MATCH_THRESHOLD} = 0.3$ (30% minimum), $\text{MAX_CANDIDATES} = 20$, $\text{sort}(S, \text{descending})$ returns sequence S sorted by normalized score in descending order, and $\text{take}(S, n)$ returns the first n elements of sequence S .

9.3 Uses

This module connects directly with:

- **Route & Trip Module** - provides trip data (origin, destination, departure time)

- **User Profile Module** - provides preference data and reliability rating.
- **Client Interaction Layer** - renders and collects user selection input.
- **Notification Module** - notifies users upon mutual matches.

It is the computational bridge between trip listings and user engagement, supporting the key matchmaking feature.

9.4 Syntax

Exported Constants

- `MAX_CANDIDATES` = 20 - limits results per query.
- `MATCH_THRESHOLD` = 0.3 - minimum normalized score (30%) to display.

Exported Access Programs

Name	In	Out	Exceptions
<code>findMatchesForUser</code>	<code>userId: N</code> , <code>request: RequestData</code>	<code>RankedMatchList</code>	<code>NotFound</code>
<code>requestRide</code>	<code>rideId: N</code> , <code>pickup: Location</code>	<code>Confirmation</code>	<code>NotFound</code> , <code>BadRequest</code>
<code>acceptRequest</code>	<code>requestId: N</code>	<code>Confirmation</code>	<code>NotFound</code> , <code>BadRequest</code> , <code>Forbidden</code>
<code>cancelRequest</code>	<code>requestId: N</code>	<code>Confirmation</code>	<code>NotFound</code> , <code>Forbidden</code>
<code>getMatchResults</code>	<code>userId: N</code>	<code>MatchSummary</code>	<code>None</code>
<code>computeScore</code>	<code>rider: UserRecord</code> , <code>driver: UserRecord</code>	<code>Score: ℝ</code>	<code>AlgorithmError</code>

9.5 Semantics

State Variables The module state is defined by the current state machine state and associated data:

- `ModuleState` $\in \{\text{UNINITIALIZED, INITIALIZING, READY, PROCESSING, IDLE, ERROR, SHUT}$
— Current lifecycle state
- `OperationState` $\in \text{MatchStates} \cup \text{RequestStates} \cup \text{AcceptStates}$
 $\cup \text{CancelStates} \cup \text{ResultsStates}$
— Current operation state,
where each set contains the states for the respective operation
- `CandidatePool` - Set of Candidate Records (drivers or riders)
(valid in states: `QUERYING_RIDES`, `VALIDATING_CANDIDATES`, `COMPUTING_ROUTES`, `SCORING_CANDIDATES`)

- **ActiveStrategy** - reference to the current scoring algorithm
(valid when $\text{ModuleState} \in \{\text{READY}, \text{PROCESSING}\}$)
- **MatchResults** - ordered sequence of potential matches with scores
(valid in states: RANKING_MATCHES , MATCH_COMPLETE)
- **OperationQueue** - queue of pending operations
(valid when $\text{ModuleState} = \text{READY}$ or PROCESSING)
- **CurrentOperation** - currently executing operation
(valid when $\text{ModuleState} = \text{PROCESSING}$)

Environment Variables

- Active network connection for queries.
- Persistence layer connection for retrieving and updating match data.
- Access to external mapping service for route distance calculations.

Assumptions

- Each user has an active, verified profile with trip data available.
- Distance and time calculations are performed via a reliable geocoding service.
- Only verified institutional users participate in matching.

Access Routine Semantics All access routines are specified as state machine transitions. Each routine transitions the module from READY state through operation-specific states to either a completion state or error state.

findMatchesForUser(userId):

- **precondition:** $\text{ModuleState} = \text{READY} \wedge \text{userId} \in \mathbb{N} \wedge \text{userId} > 0$

- **state transition sequence:**

```

READY  $\xrightarrow{\text{findMatchesForUser}(\text{userId})}$  PROCESSING
PROCESSING  $\rightarrow$  MATCH_INITIAL
MATCH_INITIAL  $\rightarrow$  FETCHING_PREFERENCES
      action: riderPrefs := queryPreferences(userId)
FETCHING_PREFERENCES  $\rightarrow$  QUERYING_RIDES
      action: CandidatePool := retrieveCandidates(userId)
QUERYING_RIDES  $\rightarrow$  VALIDATING_CANDIDATES
      action: validCandidates :=  $\{r \in \text{CandidatePool} \mid \text{availableSeats}(r) > 0\}$ 
VALIDATING_CANDIDATES  $\rightarrow$  COMPUTING_ROUTES
      action:  $\forall c \in \text{validCandidates} : \text{routeDetails}[c] := \text{getDetourAndRides}(c)$ 
COMPUTING_ROUTES  $\rightarrow$  SCORING_CANDIDATES
      action: scoredMatches := map(computeScore, validCandidates)
SCORING_CANDIDATES  $\rightarrow$  FILTERING_THRESHOLD
      action: validMatches :=  $\{m \in \text{scoredMatches} \mid \text{normalizeScore}(m.\text{score}) \geq \text{threshold}\}$ 
FILTERING_THRESHOLD  $\rightarrow$  RANKING_MATCHES
      action: MatchResults := sort(validMatches, descending)
RANKING_MATCHES  $\rightarrow$  MATCH_COMPLETE
      action: MatchResults := take(MatchResults, MAX_CANDIDATES)
MATCH_COMPLETE  $\rightarrow$  IDLE
IDLE  $\rightarrow$  READY

```

- **postcondition:** $\text{ModuleState} = \text{READY} \wedge \text{MatchResults} \neq \text{null} \wedge |\text{MatchResults}| \leq \text{MAX_CANDIDATES}$
where $\text{MAX_CANDIDATES} = 20$
- **output:** Returns MatchResults (ranked list of matches with normalized scores).
- **exception:** If transition to MATCH_ERROR occurs: NotFound
(when $\text{CandidatePool} = \emptyset$ or $\text{validCandidates} = \emptyset$ or $\text{validMatches} = \emptyset$).
Error recovery transitions to READY state.

`requestRide(rideId, pickupLat, pickupLng):`

- **precondition:** $\text{ModuleState} = \text{READY} \wedge \text{rideId} \neq \text{null} \wedge \text{pickupLat}, \text{pickupLng} \in \mathbb{R}$

- **state transition sequence:**

```

READY  $\xrightarrow{\text{requestRide}}$  PROCESSING
PROCESSING  $\rightarrow$  REQUEST_INITIAL
REQUEST_INITIAL  $\rightarrow$  VALIDATING_RIDE
                        action: ride := queryRide(rideId)
VALIDATING_RIDE  $\rightarrow$  CHECKING_SEATS
                        precondition: ride  $\neq$  null
CHECKING_SEATS  $\rightarrow$  CREATING_REQUEST
                        precondition: ride.bookedSeats < ride.maxSeats
                        action: insert(rideRequests, {rideId, riderId, pickupLat, pickupLng, status})
CREATING_REQUEST  $\rightarrow$  REQUEST_COMPLETE
REQUEST_COMPLETE  $\rightarrow$  IDLE  $\rightarrow$  READY

```

- **postcondition:** $\text{ModuleState} = \text{READY} \wedge \exists r \in \text{rideRequests} : r.\text{id} = \text{requestId} \wedge r.\text{status} = \text{"pending"}$
- **output:** Confirmation with requestId.
- **exception:** If transition to REQUEST_ERROR occurs: NotFound (ride not found) or BadRequest (ride full).

`acceptRequest(requestId):`

- **precondition:** $\text{ModuleState} = \text{READY} \wedge \text{requestId} \neq \text{null}$

- **state transition sequence:**

$$\text{READY} \xrightarrow{\text{acceptRequest}} \text{PROCESSING}$$

$$\text{PROCESSING} \rightarrow \text{ACCEPT_INITIAL}$$

$$\text{ACCEPT_INITIAL} \rightarrow \text{VALIDATING_REQUEST}$$

$$\text{action: request := queryRequest(requestId)}$$

$$\text{VALIDATING_REQUEST} \rightarrow \text{VERIFYING_DRIVER}$$

$$\text{precondition: request.status = "pending"}$$

$$\text{VERIFYING_DRIVER} \rightarrow \text{CHECKING_CAPACITY}$$

$$\text{precondition: ride.driverId = ctx.userId}$$

$$\text{CHECKING_CAPACITY} \rightarrow \text{UPDATING_STATUS}$$

$$\text{precondition: ride.bookedSeats < ride.maxSeats}$$

$$\text{action: request.status := "accepted"}$$

$$\text{UPDATING_STATUS} \rightarrow \text{INCREMENTING_SEATS}$$

$$\text{action: ride.bookedSeats := ride.bookedSeats + 1}$$

$$\text{INCREMENTING_SEATS} \rightarrow \text{ACCEPT_COMPLETE}$$

$$\text{ACCEPT_COMPLETE} \rightarrow \text{IDLE} \rightarrow \text{READY}$$

- **postcondition:** $\text{ModuleState} = \text{READY} \wedge \text{request.status} = \text{"accepted"} \wedge \text{ride.bookedSeats} = \text{ride.bookedSeats}_{\text{old}} + 1$
- **output:** Success confirmation.
- **exception:** If transition to `ACCEPT_ERROR` occurs: `NotFound`, `BadRequest`, or `Forbidden`.

`cancelRequest(requestId):`

- **precondition:** $\text{ModuleState} = \text{READY} \wedge \text{requestId} \neq \text{null}$

- **state transition sequence:**

READY $\xrightarrow{\text{cancelRequest}}$ PROCESSING
 PROCESSING $\rightarrow$ CANCEL_INITIAL
 CANCEL_INITIAL $\rightarrow$ VALIDATING_OWNERSHIP
 precondition: request.riderId = ctx.userId
 VALIDATING_OWNERSHIP $\rightarrow$ CHECKING_STATUS
 CHECKING_STATUS $\rightarrow$ UPDATING_CANCEL
 action: request.status := "cancelled"
 UPDATING_CANCEL $\xrightarrow{\text{was accepted}}$ DECREMENTING_SEATS
 action: ride.bookedSeats := max(0, ride.bookedSeats - 1)
 UPDATING_CANCEL $\xrightarrow{\text{was pending}}$ CANCEL_COMPLETE
 DECREMENTING_SEATS $\rightarrow$ CANCEL_COMPLETE
 CANCEL_COMPLETE $\rightarrow$ IDLE $\rightarrow$ READY

- **postcondition:** ModuleState = READY $\wedge$ request.status = "cancelled"
- **output:** Success confirmation.
- **exception:** If transition to CANCEL_ERROR occurs: NotFound or Forbidden.

getMatchResults(userId):

- **precondition:** ModuleState = READY $\wedge$ userId $\in \mathbb{N}$
- **state transition sequence:**

READY $\xrightarrow{\text{getMatchResults}}$ PROCESSING
 PROCESSING $\rightarrow$ RESULTS_INITIAL
 RESULTS_INITIAL $\rightarrow$ QUERYING_REQUESTS
 action: acceptedRequests := query(rideRequests,
 {riderId = userId, status = "accepted"})
 QUERYING_REQUESTS $\xrightarrow{\text{requests found}}$ FETCHING_DRIVERS
 action: $\forall r \in \text{acceptedRequests} :$
 $r.\text{driver} := \text{queryUser}(r.\text{driverId})$
 QUERYING_REQUESTS $\xrightarrow{\text{no requests}}$ RESULTS_COMPLETE
 FETCHING_DRIVERS $\rightarrow$ RESULTS_COMPLETE
 RESULTS_COMPLETE $\rightarrow$ IDLE $\rightarrow$ READY

- **postcondition:** $\text{ModuleState} = \text{READY} \wedge \text{results} \neq \text{null}$
- **output:** Returns list of confirmed mutual matches (may be empty).
- **exception:** None (returns empty list if no matches found).

`computeScore(rider, driver):`

- **precondition:** $\text{ModuleState} = \text{READY} \vee (\text{ModuleState} = \text{PROCESSING} \wedge \text{OperationState} = \text{SCORING_CANDIDATES})$
- **transition:** Pure function, no state modification. Computes score using `ActiveStrategy`.
- **output:** Floating-point score between 0 and 100 computed as $\text{normalizeScore}(\text{rawScore}(r, d))$ where `rawScore` is defined by Equation (3) and `normalizeScore` is defined by Equation (4).
- **exception:** `AlgorithmError` if scoring function fails (e.g., `ActiveStrategy` is undefined).

Local Functions

- `applyFilters(user, candidate)` - evaluates eligibility constraints (status="scheduled", seat availability, compatibility dealbreakers).
- `calculateWeightedScore(rider, driver)` - computes weighted sum using factors (default preset):
 - Schedule score (weight 2.0) - time compatibility
 - Location score (weight 2.0) - route/detour compatibility
 - Cost score (weight 1.5) - cost attractiveness
 - Comfort score (weight 0.5) - seat availability
 - Compatibility score (weight 1.0) - preference matching
- `normalizeScore(raw)` - scales raw score (0-7.0) into $[0, 100]$ using formula: $\min(100, \lfloor (\text{rawScore}/7.0) \times 100 + 0.5 \rfloor)$, which rounds to the nearest integer.

Scheduling Module

10.1 Module

The Scheduling Module manages recurring and one-time ride schedules for drivers and riders. It allows users to automate trip creation, define weekly patterns, and ensures that recurring schedules generate valid trip entries. This module integrates directly with the Trip Module and the Notification Module.

Frontend Implemented in the Expo app, the frontend provides UI controls for defining recurring or one-time schedules.

Key capabilities:

- Time picker for selecting departure time
- Day-of-week toggles for recurring schedules
- Interface to view and cancel existing schedules
- Integration with tRPC hooks such as `useCreateSchedule`, `useGetUserSchedules`
- Form validation for time ranges and required fields

Backend The backend implements scheduling logic via tRPC.

Responsibilities include:

- Parsing user-defined recurrence patterns
- Generating future trips based on schedules
- Ensuring schedules do not conflict with existing trips
- Validating schedule timing (future date, valid days)
- Exposing procedures such as `createSchedule`, `cancelSchedule`

Data The module persists schedule definitions and generated trip associations.

Tables:

- `schedules` — recurring schedule definitions (days, time, `userId`)
- `schedule_instances` — maps schedules to generated trips
- `trips` — created by this module but stored by Trip Module

10.2 Uses

- Trip Module — creates trip entries from schedules
- User Profile Module — determines valid schedule types
- Notification Module — sends reminders for upcoming rides

10.3 Syntax

Exported Constants

- `MAX_RECURRING_YEARS = 1` — Maximum duration in years to generate future instances.
- `VALID_DAYS = {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday}`

Exported Access Programs

Name	In	Out	Exceptions
<code>createSchedule</code>	<code>userId: $\mathbb{N}$, data: ScheduleData</code>	<code>ScheduleRecord</code>	<code>ValidationError</code>
<code>getUserSchedules</code>	<code>userId: $\mathbb{N}$</code>	<code>ScheduleList</code>	<code>NotFoundError</code>
<code>cancelSchedule</code>	<code>userId: $\mathbb{N}$, scheduleId: $\mathbb{N}$</code>	<code>Confirmation</code>	<code>NotFoundError, UnauthorizedError</code>
<code>generateFutureTrips</code>	<code>scheduleId: $\mathbb{N}$</code>	<code>TripList</code>	<code>AlgorithmError</code>

10.4 Semantics

State Variables

- **Schedules:** Map of $(\mathbb{N} \rightarrow \text{ScheduleRecord})$ — The collection of all schedule definitions indexed by `scheduleId`.
- **ActiveRules:** Map of $(\mathbb{N} \rightarrow \text{string})$ — The collection of parsed recurrence rules (e.g., cron strings) indexed by `scheduleId`.

Environment Variables

- System clock and device timezone for temporal calculation and validation.
- Persistence layer connection (Database).

Assumptions

- The user is authenticated, and their `userId` is verified by the Authentication Module prior to invocation.
- The database schema is fully migrated and accessible.

Access Routine Semantics `createSchedule(userId, data):`

- **transition:** Generates a unique `id`. Formats `newSchedule` combining `id`, `userId`, and `data`. Updates state:
 - `Schedules[id] := newSchedule`
 - `ActiveRules[id] := parseRecurrenceRule(data)`
- **output:** Returns `Schedules[id]`.
- **exception:** `ValidationError` if `data.isRecurring = true ∧ data.daysOfWeek = ∅`, or if `data.time` is in the past relative to the system clock.

`getUserSchedules(userId):`

- **transition:** None.
- **output:** Returns the sequence $\{s \in \text{Schedules} \mid s.\text{userId} = \text{userId}\}$.
- **exception:** `NotFoundError` if the resulting sequence is empty.

`cancelSchedule(userId, scheduleId):`

- **transition:** Removes `scheduleId` from `Schedules` and `ActiveRules`. Notifies dependent database constraints to cascade status changes.
- **output:** Returns `true` upon successful cancellation.
- **exception:** `NotFoundError` if `scheduleId ∉ Schedules`. `UnauthorizedError` if `Schedules[scheduleId].userId ≠ userId`.

`generateFutureTrips(scheduleId):`

- **transition:**
 - Retrieves rule $r = \text{ActiveRules}[\text{scheduleId}]$.
 - Uses `nextOccurrence` to compute dates.
 - Generates `TripRecord` entries via the Trip Module up to `MAX_RECURRING_YEARS`.
- **output:** Returns the generated sequence of `TripRecord` entities.
- **exception:** `AlgorithmError` if `scheduleId ∉ ActiveRules` or if the string rule fails to parse into valid future dates.

Local Functions

- `parseRecurrenceRule(data: ScheduleData): string` — Converts schedule days and time into a cron-formatted string or scheduling equivalent.
- `nextOccurrence(rule: string, after: ℝ): ℝ` — Computes the next valid Unix timestamp given a recurrence rule and a baseline timestamp.

11 Notification Module

11.1 Module

The Notification Module handles system alerts including match notifications, trip updates, and reminders.

Frontend Implemented via Expo Notifications.

UI responsibilities:

- Requesting push permission
- Displaying notification center
- Handling tap events

Frontend integration:

- `useRegisterPushToken`
- `useGetNotifications`

Backend Backend procedures manage dispatch and retrieval:

- Storing device push tokens
- Sending push messages via Expo Push API
- Queueing async notification events
- Procedures: `sendNotification`, `markAsRead`

Data

- `notifications` — stored messages
- `push_tokens` — device tokens

11.2 Uses

- Matching Module — notifies mutual matches
- Scheduling Module — sends reminders
- Trip Module — alerts cancellations

11.3 Syntax

Exported Access Programs

Name	In	Out	Exceptions
<code>sendNotification</code>	<code>userId: $\mathbb{N}$, message: string</code>	<code>Confirmation</code>	<code>DeliveryError</code>
<code>getNotifications</code>	<code>userId: $\mathbb{N}$</code>	<code>NotificationList</code>	<code>NotFoundError</code>
<code>markAsRead</code>	<code>userId: $\mathbb{N}$, notificationId: $\mathbb{N}$</code>	<code>Confirmation</code>	<code>NotFoundError</code> , <code>UnauthorizedError</code>

11.4 Semantics

State Variables

- **Notifications:** Map of $(\mathbb{N} \rightarrow \text{NotificationRecord})$ — The collection of all generated system notifications indexed by `notificationId`.
- **PushTokens:** Map of $(\mathbb{N} \rightarrow \text{string})$ — The collection of active Expo push tokens indexed by `userId`.

Environment Variables

- External Expo Push API service for outbound message delivery.
- Persistence layer connection (Database) to store notification history.

Assumptions

- The user has granted operating system-level permissions to receive push notifications on their device.
- The external Expo Push API is available and responsive.
- The `userId` passed to the access programs is authenticated.

Access Routine Semantics `sendNotification(userId, message):`

- **transition:**
 - Generates a unique `id`. Creates a new `NotificationRecord` and sets `Notifications[id] := new record`.
 - Retrieves the user's push token from `PushTokens[userId]`.
 - Dispatches the message payload to the Expo Push API via the `enqueueNotification` local function.

- **output:** Returns `true` if the notification is successfully generated and queued for delivery.
- **exception:** `DeliveryError` if the user does not have a registered push token in `PushTokens`, or if the external Expo Push service rejects the payload.

`getNotifications(userId):`

- **transition:** `None`.
- **output:** Returns the sequence $\{n \in \text{Notifications} \mid n.\text{userId} = \text{userId}\}$ sorted by timestamp descending.
- **exception:** `NotFoundError` if the resulting sequence is empty.

`markAsRead(userId, notificationId):`

- **transition:** Updates the read status of the specified notification: `Notifications[notificationId].true`.
- **output:** Returns `true` upon successful status update.
- **exception:** `NotFoundError` if `notificationId` $\notin$ `Notifications`. `UnauthorizedError` if `Notifications[notificationId].userId` $\neq$ `userId`.

Local Functions

- `formatNotificationPayload(message: string, token: string): JSON` — Constructs the strictly formatted JSON payload required by the Expo Push API endpoint.
- `enqueueNotification(payload: JSON): bool` — Appends the formatted payload to the asynchronous delivery queue and initiates the network request.

12 Rating & Feedback Module

12.1 Module

This module collects and stores ride ratings and feedback, updating reliability scores used by the Matching Module.

Frontend UI elements:

- Post-ride rating screen
- Optional text feedback box
- History of past ratings

Hooks:

- `useSubmitRating`
- `useGetUserRatings`

Backend Backend responsibilities:

- Validating rating payloads
- Linking ratings to matches
- Recomputing reliability scores
- Procedures: `submitRating`, `calculateReliability`

Data Tables:

- `ratings`
- `reliability_metrics`

12.2 Uses

- Matching Module — reliability score as weighted factor
- User Profile Module — shows aggregated rating

12.3 Syntax

Exported Access Programs

Name	In	Out	Exceptions
<code>submitRating</code>	<code>ratingData</code>	<code>Confirmation</code>	<code>ValidationError</code>
<code>getUserRatings</code>	<code>userId</code>	<code>RatingList</code>	<code>NotFoundError</code>
<code>calculateReliability</code>	<code>userId</code>	<code>Score</code>	<code>AlgorithmError</code>

12.4 Semantics

State Variables

- `recentRatings`
- `reliabilityScore`

Local Functions

- `computeWeightedAverage`
- `sanitizeComment`

13 Safety & Reporting Module

13.1 Module

This module manages user-submitted safety reports and flags, assisting platform moderation.

Frontend Includes:

- Report User form
- Safety resources page

Hooks:

- `useSubmitReport`

Backend Backend responsibilities:

- Validating and storing incident reports
- Assigning severity scores
- Procedures: `submitReport`, `resolveReport`

Data Tables include:

- `reports`
- `safety_flags`

13.2 Uses

- Admin & Moderation Module
- Notification Module

13.3 Syntax

Exported Access Programs

Name	In	Out	Exceptions
<code>submitReport</code>	<code>reportData</code>	<code>Confirmation</code>	<code>ValidationError</code>
<code>getReports</code>	<code>userId/adminId</code>	<code>ReportList</code>	<code>UnauthorizedError</code>
<code>resolveReport</code>	<code>reportId, action</code>	<code>Confirmation</code>	<code>PermissionError</code>

13.4 Semantics

State Variables

- `pendingReports`
- `userSafetyStatus`

Local Functions

- `categorizeSeverity`
- `flagUser`
- `closeReport`

14 Payment Module

14.1 Module

This module is responsible for handling in-app user payments. The implementation of this module will be done through Stripe. This acts as a payment gateway to ensure a secure transaction and successful payment.

Frontend The frontend of this application will be implemented using the Expo Client Application. A payment screen would display a form to fill out the required information for processing the payment. This includes the user's full name, mailing address, payment method, and payment information (i.e., Credit Card Number, etc.). For other methods of payment (i.e., Google Pay, PayPal, etc.), the frontend will be implemented to manage redirection. Upon a successful transaction, the user can select an option to go back to the home screen.

Backend The backend will receive the inputs from frontend and validate the user and ride data. It is then responsible for processing the payment via calling payment gateway API to authorize the payment method and details. It will open a secure session to process the payment and get the state (i.e., payment successful, processing, error) to display as the output. Lastly, this component will ensure that the transaction is recorded in the database and that receipts have been sent to the user's emails.

Data The database component is responsible for storing the user's payment history, ride details, and transaction information.

14.2 Uses

This module ensures secure access to Hitchly by verifying each user's McMaster University affiliation. It interacts directly with:

- **User Profile Module:** This is used to retrieve user profile details for verification.
- **Route & Trip Module:** This is used to retrieve trip details like cost, ID, etc.
- **Database Module:** This is used to store trip transaction details into the database.

14.3 Syntax

Exported Constants

- `max_attempt = 4` Maximum attempts to make a transaction before they have to wait for 30 minutes.
- `max_session = 25` Maximum allowed time (in minutes) for a session to process the payment before it throws a processing error.

Exported Access Programs

Name	In	Out	Exceptions
<code>init_payment</code>	<code>user_id</code> , <code>ride_id</code>	Boolean (verified or unverified)	<code>NotFoundError</code>
<code>process_payment</code>	<code>payment_id</code> , <code>payment_method</code>	<code>payment_status</code>	<code>ExpiredSessionError</code>
<code>get_paymentRecord</code>	<code>payment_id</code> , <code>user_id</code> , <code>ride_id</code>	<code>paymentRecord</code>	<code>NotFoundError</code>

14.4 Semantics

State Variables

- `current_amount`: `float` — holds the current amount for the transaction.

- **session_token:** `string` — holds the current amount for the transaction.
- **sessionState:** `object` — holds session metadata for frontend persistence.

Environment Variables

- Secure HTTPS connection to API.
- Secure API connection to payment gateways.

Assumptions

- The users won't intentionally input expired or invalid credentials.
- The API connection is secure and stable throughout the payment process.
- The payment gateways will handle user's confidential information securely.

Access Routine Semantics `init_payment:`

- **transition:** None
- **output:** `Bool` (user verified for the specific ride or not)
- **exception:** `NotFoundError`, user or ride not found.

`Process_payment:`

- **transition:** Initializes connection with the API.
- **output:** Returns whether the payment is succesful, processing or failed.
- **exception:** `ExpiredSessionError` if the session fails to process the payment due to session limits.

`get_paymentRecord:`

- **transition:** Creates a payment record.
- **output:** List of payment records.
- **exception:** `NotFoundError` if one of the required parameters is missing.

Local Functions

- **generateVerificationToken:** `string` — Creates a cryptographically secure token.

15 Database Module

15.1 Module

This module is responsible for handling all database-related functionalities and ensuring smooth communication between the database servers for retrieving and updating data. This includes user information, ride details, transaction logs, etc.

Frontend N/A

Backend The backend layer of this module handles CRUD operations for core system entities and manages communication between the application and the database for smooth retrieval and update of application data.

Data This layer will store a schema of tables used across multiple modules. This includes:

- **User Table:** This stores all critical profile information of the user.
- **Ride Table:** This stores all critical ride details.
- **Payment Table:** This stores the user transactions for each ride.
- **Safety Report Table:** This stores all user safety reports.
- **Admin Table:** This stores the user's role details.

15.2 Uses

- **User Profile Module:** This is used to retrieve and store user profile details.
- **Route & Trip Module:** This is used to retrieve and store trip details like cost, ID, etc.
- **Payment Module:** This is used to retrieve and store user's transaction details.
- **Safety and Reporting Module:** This is used to retrieve and store user reports.
- **Admin Module:** This is used to retrieve and store admin details.

15.3 Syntax

Exported Constants

- `max_connect = 30` This is the maximum number of database connections at a time.
- `max_attempt = 4` Maximum attempts made to connect to db before the connection fails.
- `max_DB_session = 30` 30 minutes limit before the session times out.

Exported Access Programs

Name	In	Out	Exceptions
addRecord	tablename, data	Boolean	NotFoundError, InvalidDataFormatException, DBTimeoutException
updateRecord	tablename, data, record_id	Boolean	NotFoundError, InvalidDataFormatException, DBTimeoutException
deleteRecord	tablename, record_id	Boolean	NotFoundError, InvalidDataFormatException, DBTimeoutException
getRecord	tablename, data_condition	List of data records	NotFoundError, InvalidDataFormatException, DBTimeoutException
getAllRecords	tablename	List of data records	NotFoundError, InvalidDataFormatException, DBTimeoutException

15.4 Semantics

State Variables

- **DBConnections:** `List<connections>` — List of active database connections.

Environment Variables

- Secure HTTPS connection to API.
- Secure API connection and communication the database server.
- Secure connection to local/cloud backup services for data backup.

Assumptions

- The Postgress database is avaiable and securely stores data.
- The database has sufficient storage capacity.
- The database automatically runs its data backup services to ensure that no data is lost.

Access Routine Semantics `addRecord`:

- **transition:** A new entry is added to the specified table.
- **output:** `Bool` (True or False, depending on the state of the transition).
- **exception:** `NotFoundError` (table not found), `InvalidDataFormatException`, `DBTimeoutException`.

`updateRecord`:

- **transition:** An existing entry is updated in the specified table.
- **output:** `Bool` (True or False, depending on the state of the transition).
- **exception:** `NotFoundError` (table or entry not found), `InvalidDataFormatException`, `DBTimeoutException`.

`deleteRecord:`

- **transition:** An existing entry is deleted in the specified table.
- **output:** Bool (True or False, depending on the state of the transition).
- **exception:** `NotFoundError` (table or entry not found), `InvalidDataFormatException`, `DBTimeoutException`.

`getRecord:`

- **transition:** None.
- **output:** Returns a list of indicated records.
- **exception:** `NotFoundError` (table not found), `DBTimeoutException`.

`getAllRecords:`

- **transition:** None.
- **output:** Returns a list of all records.
- **exception:** `NotFoundError` (table not found), `DBTimeoutException`.

Local Functions

- **db_connect:** `bool` — This is to initialize a connection with the database service.
- **checkFormat:** `bool` — This function checks if the inserted data aligns with the specified schema.

16 Pricing Module

16.1 Module

This module handles the initial cost estimation functionality of the application.

Frontend This layer provides the UI for displaying the factors influencing the price per ride, along with a price estimation for it. It dynamically updates the price as new rides join the ride for a specific ride.

Backend This layer provides the algorithm to use those factors and accurately calculate the cost estimation for each rider for a given trip. It interacts with the Google Maps API to fetch location and distance values. Additionally, it sends this data to the payment module for processing the final payment for each rider.

Data The database component is responsible for storing the following table:

- **Pricing Table:** Contains data related to pricing as per the parameters.
- **Ride Table:** Contains data related to ride details.

16.2 Uses

- **User Profile Module:** This is used to retrieve the user type.
- **Database Module:** This is used to retrieve and store pricing details.
- **Route & Trip Module:** This is used to retrieve and store trip details like cost, ID, etc.
- **Payment Module:** The final cost estimation from this module is sent to the payment module.

16.3 Syntax

Exported Constants

- `max_session_time` = 30 30 minutes limit before the API session times out.

Exported Access Programs

Name	In	Out	Exceptions
<code>get_ride_summary</code>	<code>rideID</code>	<code>ride_summary</code>	<code>NotFoundError</code>
<code>estimate_cost</code>	<code>start.location, end.location, rider.count</code>	<code>fare_estimate</code>	<code>InvalidLocationException, APITimeOutException</code>
<code>update_fare</code>	<code>new_rider.count</code>	<code>updated_fare</code>	<code>APITimeOutException</code>

16.4 Semantics

State Variables N/A

Environment Variables

- Secure HTTPS connection to API.
- Secure Maps API connection for accurate estimation of travel time and distance.
- Secure API connections to get daily fuel rates.

Assumptions

- The external API services function properly with minimal downtime.
- The fuel estimates online are accurate.
- The internet connection stays stable during the cost estimation process.

Access Routine Semantics `get_ride_summary`:

- **transition:** None.
- **output:** Returns a ride summary object.
- **exception:** `NotFoundError` (ride not found).

`estimate_cost`:

- **transition:** Retrieves information from API and updates database tables.
- **output:** `Fare_estimate`, returns an estimated price value.
- **exception:** `InvalidLocationException`, `APITimeOutException`.

`update_fare`:

- **transition:** Retrieves information from API and updates database tables.
- **output:** `Updated_fare`, returns an estimated price value.
- **exception:** `APITimeOutException`.

Local Functions

- `validate_parameters(start_location, end_location, rider_count): bool` — Check if the values are not negative and are valid inputs.

17 Admin/Moderation Module

17.1 Module

This module handles the distribution of user roles and management of admin controls. This also controls and monitors activity and ensures strict policy enforcement.

Frontend This layer is responsible for providing an admin dashboard with flagged content, user complaints, user roles, and application analytics.

Backend This layer is responsible for handling communication with RESTAPI endpoints using AdminRouter to manage user accounts and rides. Additionally, it runs the logic for handling role-based authentication.

Data

- **Admin Table:** This would include profile information about admin.
- **Ride Table:** This would include information about users with warnings/bans.
- **User Analytics Table:** This includes a dataset of user and application statistics.

17.2 Uses

- **Database Module:** This is used to retrieve and store pricing details.
- **Authentication & Verification Module:** This is used to manage admin login and permissions.

17.3 Syntax

Exported Constants

- `warning_threshold= 3` Maximum warnings before putting an account ban.

Exported Access Programs

Name	In	Out	Exceptions
<code>warn_user</code>	<code>user_ID, reason</code>	Bool (check if user is successfully warned)	<code>NotFoundError</code> , <code>InsufficientAdminPrivileges</code>
<code>get_Reports</code>	<code>None</code>	Returns a dataset of user reports	<code>NotFoundError</code> , <code>InsufficientAdminPrivileges</code>
<code>get_analytics</code>	<code>None</code>	Returns a dataset of user and application statistics	<code>NotFoundError</code> , <code>InsufficientAdminPrivileges</code>

17.4 Semantics

State Variables N/A

Environment Variables

- Secure HTTPS connection to API.
- Secure API connection for stable admin controls and management.

Assumptions

- All admin permissions function properly.
- All database services are full functional and stable.
- All API connections are functional and stable.

Access Routine Semantics `warn_user`:

- **transition:** Adds a new record with a warning in the reports table.
- **output:** Bool (check if user is successfully warned).
- **exception:** `NotFoundError` (user_ID not found), `InsufficientAdminPrivileges`.

`get_Reports`:

- **transition:** None.
- **output:** Returns a dataset of user reports.
- **exception:** `NotFoundError` (empty dataset), `InsufficientAdminPrivileges`.

`get_analytics`:

- **transition:** None.
- **output:** Returns a dataset of user and application statistics.
- **exception:** `NotFoundError` (empty dataset), `InsufficientAdminPrivileges`.

Local Functions

- `check_privilege(admin_ID, endpoint): bool` — This function validates admin privileges and returns a bool to indicate whether they have sufficient privileges or not.

18 Appendix

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
 - **Aidan Froggatt:** For me, the deliverable went well because I already had strong familiarity with our codebase and architectural decisions. Since I set up the initial project foundation—Turborepo structure, tRPC API, database schema, and BetterAuth configuration—I had a clear understanding of how the modules needed to be defined and integrated. This made it easier to produce accurate, consistent MIS entries. I also found that working through each module clarified how our system would scale and how responsibilities should be divided, which improved the overall coherence of the document.
 - **Swesan:** The previous documents (such as the SRS and the earlier drafts) had already established much of the conceptual groundwork, which made defining the modules relatively straightforward. Since we entered this deliverable with a strong understanding of the system's structure, identifying responsibilities and writing the Module Guide flowed naturally. I personally worked on Section 5 (Module Hierarchy) and Section 7, and having those earlier documents as references made the process efficient and focused.
 - **[Burhanuddin Kharodawala]:** We had a good discussion as a group which helped me develop a better understanding of the overall content of this deliverable. Moreover, the template had clear understandings for most of the sections I worked on.
2. What pain points did you experience during this deliverable, and how did you resolve them?
 - **Aidan Froggatt:** One of the main challenges for me was determining the correct level of abstraction for each module. Because I am so close to the actual

implementation, it was easy to accidentally include too many low-level details. I had to step back and focus on "what" the module exposes rather than "how" it works internally. Another pain point was aligning terminology with the Module Guide—for example, ensuring naming consistency and clearly defining frontend, backend, and data responsibilities. I resolved these issues by revisiting the SRS and MG structure and adjusting my MIS sections to fit the expected style and rigor of the course.

- **Swesan:** One of the main challenges was determining which modules were truly necessary and how to group or separate functionality without creating modules that were too abstract or too fragmented. Our initial draft had 15 modules, but after careful discussion and iterative refinement, we narrowed it down to 12 well-defined modules that better reflect the system's actual behaviour and design principles. We resolved this by comparing responsibilities, identifying overlaps, and merging modules where appropriate while ensuring each remaining module had a clear secret, purpose, and boundary.
 - **[Burhanuddin Kharodawala]:** The document had a few ambiguous instructions which I had difficulty understanding. The module decomposition section especially was the one thing that was a little confusing. I was unsure of what the expectations were for the module. As in, what is a module (A feature or functionality)? At the end of the TA meeting however, I was able to discuss this with the TA and confirm the expectations for the modules.
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?

As a team, many of our design choices originated from discussions with peers and user proxies. The idea to create a McMaster-only ridesharing platform emerged from user concerns around safety and trust. Our matching approach, including the swipe-based interface, was influenced by peer feedback indicating that a familiar interaction model would improve adoption. Other decisions—such as using tRPC, BetterAuth, and Drizzle ORM—came from internal technical reasoning. These choices focused on maintainability, type safety, and long-term scalability rather than direct stakeholder input.

4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), it any, needed to be changed, and why?

During the MIS development, the team identified minor inconsistencies between the Module Guide, SRS, and earlier architectural descriptions. Some module names were standardized to ensure consistent referencing, and inter-module relationships were clarified, particularly for Matching, Scheduling, and Notification. A few requirement identifiers were refined to improve traceability. No core requirements changed, but descriptions were tightened to align with the finalized MIS structure.

5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO_ProbSolutions)

The current solution, while well-structured, is limited by capstone scope and development time. Our matching algorithm is deterministic and rule-based; with more resources, the team would implement machine-learning-driven adaptive matching. Additional improvements could include:

- fully implemented CI/CD pipelines for automated testing and deployment,
- real-time route optimization using live traffic data,
- enhanced accessibility and UX refinement,
- automated moderation tools and trust scoring systems,
- end-to-end encryption and advanced safety features.

These expansions would elevate Hitchly from a functional prototype to a production-ready system.

6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design?

The team considered multiple architectural and implementation approaches. One option was to use a REST-based backend instead of tRPC, but this offered weaker type safety across the mobile app and API. Another option was building a native iOS/Android app, but Expo provided faster development and easier integration within the monorepo. The team also explored using a monolithic architecture versus a modular service structure. Ultimately, the chosen modular design offered the best balance between clarity, extensibility, and meeting course expectations. Alternative matching designs, such as a manually curated list instead of a scoring algorithm, were rejected because they reduced flexibility and diminished the quality of the user matching experience. (LO_Explores)

Module Reflection

The process of implementing the UI.Search module for Team 1 benefited significantly from the clarity of the provided MIS documentation. The specification outlined requirements for state variables, access routines, and exception types clearly, which made it straightforward to translate requirements into code. Having explicit definitions for functions like `submitQuery()`, `nextPage()`, and `exportResults()` meant there was little ambiguity about what needed to be built. The existing codebase was well-structured and followed consistent patterns throughout. The team had clear separation between services, types, components, and pages. This organization reduced the learning curve

considerably and made it easy to understand where new code should be placed. Additionally, much of the backend infrastructure including Electron IPC handlers, fragment filtering logic, and database query support was already in place, allowing the new module to build upon existing foundations rather than starting from scratch.

The most significant challenge was the absence of test data due to confidentiality constraints. The team's manuscript fragment images are sensitive research materials that cannot be included in the public repository. As a result, the data folder was empty, the database contained no fragment records. While the UI renders properly and displays appropriate empty states, end-to-end testing was difficult.

Overall, this was a great learning experience that showed the importance of writing clear requirements and specifications for an application before beginning development.

References

- Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.
- Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL <http://citeseer.ist.psu.edu/428727.html>.
- D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.